

Tutorial - 10

In this tutorial, you will design and optimize a couple of numerical methods written in OpenACC.

- Design and implement QR decomposition using modified Gram-Schmidt algorithm given below. This method takes as input a matrix $A[M][N]$, $M > N$, and produces two matrices $Q[M][N]$ and $R[N][N]$ where Q is an orthogonal matrix, and R is an upper triangular matrix such that $A = QR$. Initialize the matrix A with random floating point values. Optimize the OpenACC program for data transfer and parallelism for $M = 1500$ and $N = 1000$.

Algorithm 1: Modified Gram-Schmidt algorithm

function modified Gram-Schmidt(A)

$M, N \leftarrow \text{dimensions}(A)$

$Q \leftarrow \text{copy}(A)$

$R \leftarrow \text{zeros}(N, N)$

for $i = 0, \dots, (N - 1)$ do

$R_{i,i} \leftarrow \|Q_{:,i}\|$

$Q_{:,i} \leftarrow Q_{:,i} / R_{i,i}$

 for $j = (i + 1), \dots, (N - 1)$ do

$R_{i,j} \leftarrow Q_{:,j}^T Q_{:,i}$

$Q_{:,j} \leftarrow Q_{:,j} - R_{i,j} Q_{:,i}$

return Q, R

- Design, implement, parallelize and optimize a convolution filter in OpenACC. You are given a 1D vector of integers $A[N]$ and a filter $F[K]$, $K \ll N$ and K is odd. The filter is applied at each element in the vector (respecting the boundaries) which changes $A[i]$ appropriately. Each $A[i]$ is replaced by the summation of the products of the filter elements with neighbors of $A[i]$. Test your code for $K = 5$ and $N = 1000, 2000, 3000$. An example of input and output vectors for a particular filter are given below:

Input	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td></tr></table>	1	2	3	4	5	6	7	8	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td></tr></table>	1	2	3	4	5	6	7	8
1	2	3	4	5	6	7	8											
1	2	3	4	5	6	7	8											
Filter	<table><tr><td>3</td><td>4</td><td>5</td><td>4</td><td>3</td></tr></table>	3	4	5	4	3	<table><tr><td>3</td><td>4</td><td>5</td><td>4</td><td>3</td></tr></table>	3	4	5	4	3						
3	4	5	4	3														
3	4	5	4	3														
Filter output	<table><tr><td>6</td><td>12</td><td>20</td><td>20</td><td>18</td></tr></table>	6	12	20	20	18	<table><tr><td>9</td><td>16</td><td>25</td><td>24</td><td>21</td></tr></table>	9	16	25	24	21						
6	12	20	20	18														
9	16	25	24	21														
Output	<table><tr><td>1</td><td>2</td><td>3</td><td>76</td><td>5</td><td>6</td><td>7</td><td>8</td></tr></table>	1	2	3	76	5	6	7	8	<table><tr><td>22</td><td>38</td><td>57</td><td>76</td><td>95</td><td>114</td><td>106</td><td>86</td></tr></table>	22	38	57	76	95	114	106	86
1	2	3	76	5	6	7	8											
22	38	57	76	95	114	106	86											